个人学科教材 · 修订版

C 语言程序设计

程序构造、内存表示与算法

宋体正文 · 数学公式排版

概念、推导与讲解型例题

2026-10-16

目录

第一章 程序、数据与表达式 3

第二章 控制流 4

第三章 函数与程序结构 8

第四章 递归与数的表示 10

第五章 指针与间接访问 13

第六章 数组与存储边界 16

第七章 类型限定与数值表示 23

第八章 字符串与字符指针 25

第九章 字符串处理函数 27

第十章 交换与按位运算 30

第十一章 伪随机数与排序 33

第十二章 中位数与顺序统计 39

附录 术语与阅读资料 40

本册从可以运行的小程序开始，逐步引入类型、控制流、函数、指针和数组，再用字符串处理与排序把这些机制结合起来。每段代码的解释围绕执行时的对象变化展开；变量的名字不能代替类型和存储关系。

编排参考 K&R《C 程序设计语言》第 1 章的示例引入方法，以及第 5 章从地址到数组、从字符存储到复制函数的展开方式。示例按当前课程重写，采用显式返回类型与必要的边界检查。

第一章 程序、数据与表达式

一个计算任务必须先变成可以执行的程序。下面先用短程序建立源文件、函数、语句和输入输出之间的联系，再讨论数据类型如何决定运算结果。后续章节只是在这个结构中增加新的计算方式。

程序结构与标准输入输出

```
#include <stdio.h>

int main(void)
{
    int x = 10;
    printf("%d\n", x);
    return 0;
}
```

- `#include <stdio.h>`：引入标准输入输出功能。
- `int main(void)`：程序入口；`void` 表示这里不接收参数。
- `{ ... }`：代码块（блок кода）。
- `int x = 10;`：定义并初始化整型变量。
- `return 0;`：结束 `main`，状态码 `0` 通常表示正常结束。

常用格式说明符：

数据	printf	示例
int	%d	printf("%d", x);
unsigned int	%u	printf("%u", u);
long long	%lld	printf("%lld", n);
char 字符	%c	printf("%c", ch);
字符串	%s	printf("%s", str);
指针地址	%p	printf("%p", (void *)p);

打印地址不要用 `%d`，应使用 `%p`：

```
printf("%p\n", (void *)&x);
```

类型、声明与整数除法

类型（тип данных）决定可表示的值和可进行的操作。常见类型是 int、unsigned int、float、double、char；不要默认所有平台的 int 都是 4 字节。sizeof 的结果类型是 size_t，输出用 %zu。sizeof(char) 总是 1，但一字节的位数由 <limits.h> 中 CHAR_BIT 表示。

同一声明中逗号分开同类型变量；分号结束声明，换行本身不结束语句。

```
int a = 7, b = 2;
double r1 = a / b;          // 先整数除法，再变成 3.0
double r2 = (double)a / b;  // 先转换，得到 3.5
double r3 = (double)(a / b);// 已丢掉小数，仍为 3.0
```

整除（целочисленное деление）向零截断：-7 / 2 是 -3，-7 % 2 是 -1。转换（приведение типа）不会把已经丢失的小数补回来。除数不能为 0，INT_MIN / -1 在结果无法表示时也不合法。

scanf：地址和格式必须匹配

```
int age;
double height;
char grade;

if (scanf("%d %lf", &age, &height) != 2)
    return 1;
if (scanf(" %c", &grade) != 1)
    return 1;
```

scanf 需要目标对象的地址；返回值表示成功赋值的项目数，不能把输入失败后的未初始化变量拿来计算。printf 输出 double 用 %f（或 %lf），scanf 读 double 必须用 %lf，读 float 用 %f。上面 %c 前面的空格用于跳过空白字符，包括上一次输入留下的换行；如果题目本来要求读取空白字符，就不要加这个空格。

第二章 控制流

计算通常不只是从上到下执行一列语句。条件决定哪些语句执行，循环决定同一段计算重复多少次。分析控制流时，应跟踪条件被检查的时刻和一次迭代结束后的变量状态。

条件选择

条件语句（условный оператор）的基本形式：

```
if (条件)
{
    // 条件为真时执行
}
else
{
    // 条件为假时执行
}
```

例子：

```
int x = 5;

if (x > 10)
{
    printf("x 大于 10");
}
else
{
    printf("x 不大于 10");
}

x > 10 ?
  ├─ 真（истина）→ 执行 if
  └─ 假（ложь）  → 执行 else
```

多个互斥区间使用 `else if`：

```
int score = 75;

if (score >= 90)
{
    printf("A");
}
else if (score >= 60)
{
    printf("及格");
}
else
{
    printf("不及格");
}
```

程序从上到下判断；一旦某一项成立，后面的分支都跳过。

必须记住：

1. `else` 不能独立出现，它匹配前面尚未匹配的最近一个 `if`。 2. 即使分支里只有一条语句，也建议写 `{}`。 3. `=` 是赋值，`==` 才是比较。

```
if (x = 5)   // 把 5 赋给 x，再把 5 当作“真”；通常是错误
if (x == 5)  // 判断 x 是否等于 5
```

循环迭代

循环（цикл）：

```
int n = 25;

while (n > 0)
{
    printf("%d ", n);
    n = n - 7;
}
```

逐轮追踪：

本轮开始时 n	n > 0	输出	更新后的 n
25	真	25	18
18	真	18	11
11	真	11	4
4	真	4	-3
-3	假	不执行	不执行

最终输出：

```
25 18 11 4
```

不输出 -3，因为 while 每轮先检查条件，-3 > 0 为假后不会进入循环体。

对比 do ... while：

```
do
{
    循环体;
} while (条件);
```

do ... while 先执行一次再判断，因此循环体至少执行一次。

for、continue 和嵌套循环

```
for (int i = 0; i < 4; ++i)
{
    if (i == 2)
        continue;
    printf("%d ", i);
}
```

输出 0 1 3。for 的执行顺序是：初始化一次 → 判断条件 → 循环体 → 更新表达式 → 再判断。continue 跳过本轮剩余循环体，但仍执行 for 的更新表达式。while 中若把计数更新写在 continue 后，更新可能永远不执行，造成死循环。

语句	作用范围	俄语
break	离开最近一层循环或 switch	выход из цикла
continue	开始最近一层循环的下一轮	следующая итерация

语句	作用范围	俄语
return	结束整个当前函数	возврат из функции

嵌套循环中，内层计数器通常每次进入外层新一轮时重新初始化；内层 break 不会同时结束外层。goto 跳到同一函数中的标签，不会建立新函数调用；基础程序优先用清楚的结构化循环。

多路选择

```
int x = 1;

switch (x)
{
    case 1:
        printf("A ");

    case 2:
        printf("B ");
        break;

    case 3:
        printf("C ");
}

printf("D");
```

执行路线：

```
x == 1
→ 进入 case 1
→ 输出 A
→ 没有 break，继续进入 case 2
→ 输出 B
→ break 离开 switch
→ 继续执行 printf("D")
```

最终输出 A B D。

没有 break 而继续执行下一个 case 叫贯穿（fall-through；俄语可说 сквозное выполнение）。

- break：离开当前循环或当前 switch。
- return：结束当前函数，并可把结果返回给调用者。
- break 不是“结束整个程序”。

default（ветвь по умолчанию）用于没有任何 case 匹配时。

第三章 函数与程序结构

一段计算可以单独命名，并通过参数接收数据、通过返回值交出结果。这样，主程序表达任务的整体步骤，各个函数承担具体工作。函数接口先于函数内部实现：调用者需要知道输入、输出和对外部对象的影响。

函数接口与返回值

```
int f(int x)
{
    printf("%d ", x);
    return x + 2;
}

int main(void)
{
    int a;
    a = f(5);
    printf("%d", a);
    return 0;
}
```

执行顺序：

```
进入 main
→ 执行 a = f(5)
→ main 暂停，进入 f
→ f 的参数 x 得到 5
→ 输出 5
→ x + 2 得到 7
→ return 7，同时结束本次 f 调用
→ 回到 main，把 7 赋给 a
→ 输出 7
```

最终输出：5 7。

return x + 2; 同时做两件事：

1. 把计算结果作为返回值（возвращаемое значение）交给调用者； 2. 立即结束当前函数。

术语：

- 形式参数：формальный параметр
- 实参：аргумент
- 函数调用：вызов функции
- 返回值：возвращаемое значение

输出和返回不是一回事

printf 把信息写到输出；return 把结果交给调用者并结束函数。函数打印了 5，不代表它的返回值就是 5。

```
int square(int x)
{
    return x * x;
}

void show(int x)
{
    printf("%d\n", x);
    return;  // void 函数可提前结束，但不返回数值
}
```

square(3) 的返回值是 9；只有调用者再 printf 才能看到它。void 用在返回类型表示不返回数值，用在参数列表 (void) 表示不接收参数。它不代表“函数没有任何作用”。

作用域、存储期与调用栈

```
void f(int *p)
{
    int y = 5;
    *p = *p + y;
}

int main(void)
{
    int x = 10;
    f(&x);
    return 0;
}
```

正在执行 f 时，可用简化图理解调用栈（стек вызовов）：

```
较新的调用
┌──────────────────┐
│ f 的调用帧       │
│ p → main 的 x    │
│ y = 5            │
├──────────────────┤
│ main 的调用帧    │
│ x = 10           │
└──────────────────┘
较早的调用
```

执行 *p = *p + y; 后，x 变为 15。

当 f 返回时：

- f 的局部变量 y 生命周期（время жизни）结束；
- main 尚未结束，所以 main 的 x 仍然有效，而且值为 15。

简化的存储分类：

内容	教学模型中的位置
`main`、`f` 的机器指令	代码区（сегмент кода）
自动存储期的局部变量 `x`、`y`	通常在栈（стек）
全局变量、静态变量	静态存储区/数据区（сегмент данных）

这是帮助理解的常见模型。编译器优化后，局部变量可能只在寄存器中；标准 C 并不要求所有局部变量物理上都放在栈中。

返回局部值合法，返回局部地址不合法

```
int get_value(void)
{
    int y = 7;
    return y;       // 返回数值副本：合法
}

/* 反例，不要使用返回的指针 */
int *bad(void)
{
    int y = 7;
    return &y;      // y 结束生命周期后成为悬空指针
}
```

作用域（область видимости）指名字能在哪里使用；生命周期（время жизни）指对象存在多久，不能混为一谈。局部 `static` 对象的名字仍只在相应作用域可见，但对象在整个程序运行期间存在。

```
int counter(void)
{
    static int count = 0;
    return ++count;
}
```

分别调用 `counter` 三次，返回 1、2、3；初始化不是每次调用都重新执行。普通自动局部变量和局部 `static` 不能套用同一个生命周期结论。

第四章 递归与数的表示

有些任务可以分解成一个规模更小的同类任务。递归函数正是把这种分解写进函数调用中。分解必须趋近一个可以直接处理的终止情形；调用返回时，尚未完成的工作再依次继续。进制转换同时展示了递归与迭代两种组织计算的方法。

递归计算

递归（рекурсия）是函数直接或间接调用自己。

```
void f(int n)
{
    if (n > 0)
    {
        printf("%d ", n);
        f(n - 2);
        printf("%d ", n);
    }
}
```

调用 `f(5)`。

向下调用：

```
f(5)：先输出 5，再调用 f(3)，等待
f(3)：先输出 3，再调用 f(1)，等待
f(1)：先输出 1，再调用 f(-1)，等待
f(-1)：n > 0 为假，直接返回
```

向上返回：

```
回到 f(1)：执行递归调用后面的 printf，输出 1
回到 f(3)：输出 3
回到 f(5)：输出 5
```

完整输出：

```
5 3 1 1 3 5
```

最关键的规律：

```
递归调用前面的代码：深入时执行
递归调用后面的代码：返回时执行
```

递归必须有停止条件，也叫基本情况（базовый случай）。没有正确的停止条件会不断调用，最终通常导致栈溢出（переполнение стека）。

做递归题时固定画两列：

深入阶段	返回阶段
当前参数是多少	回到哪一层
调用前输出什么	调用后输出什么
下一次调用参数是什么	本层何时结束

进制转换

进制（система счисления）；余数（остаток）；数位（цифра）；位权（вес разряда）。

递归：先处理高位，再输出当前余数

```
/* 调用前保证 base 在 2..32；n 为非负数 */
void print_base(unsigned int n, unsigned int base)
{
    static const char digits[] = "0123456789ABCDEFGHIJKLMNOPQRSTUV";
    if (n >= base)
        print_base(n / base, base);
    putchar(digits[n % base]);
}
```

print_base(38u, 2u) 调用链为 38→19→9→4→2→1，返回时依次输出 1、0、0、1、1、0，得到 100110。n=0 时输出一个 0。base 的检查必须由调用者完成，不能传入 0 或 1。

和“打印进入阶段与返回阶段”的题目比较：递归前打印，会从低层调用之前执行；递归后打印，等更深一层返回才执行。若改写为每层先取余数再打印再调用，通常会得到低位在前的反向表示。

不用数组：找到最高位权，再逐位取出

```
void print_digits(unsigned int n, unsigned int base)
{
    if (base < 2u || base > 32u)
        return;

    unsigned int weight = 1u;
    while (weight <= n / base)
        weight *= base;

    while (weight > 0u)
    {
        printf("%u", n / weight);
        n %= weight;
        weight /= base;
        if (weight > 0u)
            putchar(' ');
    }
}
```

这里只输出“用十进制数字写出的每个数位”，不是 A、B、C 等字符形式。因此 63 的 32 进制显示为 1 31；10000 的 16 进制显示为 2 7 1 0。若要求标准字符形式，使用上一节 digits 查表。

weight <= n/base 避免先计算 weight*base 导致的溢出。第二个循环必须按 weight > 0 判断，不能按 n > 0：余数可能已变成 0，但低位的零仍需输出。例如 10000 的末位 0 不可丢。n=0 时 weight 初值 1，仍能打印一个 0。

带返回值的递归：输出与计算分开追踪

```
/* 前提：1 <= n <= 4，避免越界输入与整数溢出 */
int f(int n)
{
    if (n == 1)
        return 2;
    return n * f(n - 1);
}
```

f(1)=2 → f(2)=4 → f(3)=12 → f(4)=48。没有 printf 就不会自动显示 48。该递归不适合直接传 n=0；应先按函数前提检查输入。

第五章 指针与间接访问

函数参数接收的是值的副本。如果函数需要修改调用者的对象，就必须获得访问该对象的方法。对象的地址提供了这种方法，指针类型则说明从该位置访问什么类型的数据。先从单个对象的地址与解引用出发，再把相同的机制用于连续存放的数组。

指针、地址与间接访问

```
int x = 10;
int *p = &x;
```

假设 x 的地址是 1000：

```
地址 1000
┌──────────┐
│    10    │  ← x 保存的值
└──────────┘
     ↑
     p 保存 1000，所以 p 指向 x
```

表达式	含义	假设值
x	x 的值	10
&x	x 的地址	1000
p	指针 p 保存的地址	1000
*p	去 p 保存的地址处取值	10

因此：

```
p  == &x;  // 地址相等
*p == x;   // 值相等
```

同一个 * 的两种语法作用

```
int *p = &x;
```

这里的 * 是声明的一部分：声明 p 是“指向 int 的指针”。

```
printf("%d", *p);
```

这里的 * 是解引用运算符（оператор разыменования）：访问 p 指向的对象。

记忆句：

```
&：从变量得到地址
*：从地址找到对象
```

俄语课堂常见说法：

- 取地址运算符：оператор взятия адреса
- 解引用运算符：оператор разыменования
- 指向：указывает на
- p 指向 x：p указывает на x

通过指针修改变量

```
int x = 20;
int *p = &x;
*p = 35;
```

*p = 35 不是把 p 改成地址 35，而是把 p 指向的对象改成 35。因为该对象就是 x，所以最后 x == 35。

&p、指针自增和对象自增

如果 int *p = &x，那么 &p 是指针变量 p 自己的地址，类型是 int **；它不是 x 的地址。

表达式	解析与作用
p++	移动指针；用于合法的数组范围
(*p)++	指针不动，所指的整数加 1
++*p	等同 ++(*p)，先增加所指的整数
*p++	等同 *(p++)，取得原位置对象，再移动指针
*++p	等同 *(++p)，先移动再解引用新位置

不要把这些操作混在复杂的一行中。先确认指针有效、所指对象可写以及移动后的范围，再分析运算符优先级。NULL 表示空指针，不是“指向数值 0 的指针”，不能解引用。

函数参数的值传递

C 语言严格来说只有按值传递（передача по значению）。所谓“传地址”，也是把地址这个值复制给函数的指针参数。

普通值参数

```
void f(int x)
{
    x = x * 2;
}

int main(void)
{
    int x = 6;
    f(x);
    printf("%d", x);
}

main_x = 6
   ↓ 复制数值 6
f_x = 6
   ↓
f_x = 12

main_x 仍然是 6
```

最终输出 6。

指针参数

```
void f(int *p)
{
    *p = *p * 2;
}

int main(void)
{
    int x = 6;
    f(&x);
    printf("%d", x);
}

&x = 1000
→ f(&x) 把地址 1000 复制给参数 p
→ p == 1000
→ *p 就是地址 1000 里的 x
→ *p = *p * 2
→ x = 12
```

最终输出 12。

调用	传入内容	能否借此修改外部 x
f(x)	x 当前值的副本	不能
f(&x)	x 地址的副本	能，通过 *p

“传地址”可说 передача адреса 或 передача указателя；更严格的说法是 указатель передаётся по значению（指针也是按值传递）。

两个形参可能指向同一个对象

```
void change(int *p, int *q)
{
    *p += 2;
    *q *= 3;
}

/* 调用片段 */
int x = 4;
change(&x, &x);
```

结果是 18，不是分别算出 6 和 12。p、q 都指向 x：第一句把 x 改成 6，第二句读取新的 6，再乘 3。这个现象叫别名（алиасинг）。遇到两个指针参数时，先问“是不是同一个对象”，再画值变化表。

指针的算术运算

```
p + 1 的实际字节位移 = sizeof(*p)
```

在常见的 `sizeof(int) == 4` 的环境中：

```
int *pa: 1000 → pa + 1 → 1004
char *pc: 1000 → pc + 1 → 1001
```

但不要把“`int` 一定是 4 字节”当成标准规定。标准只保证 `sizeof(char) == 1`；应使用当前实现的 `sizeof(int)`。

```
a++;   // a 保存的数值加 1
pa++;  // pa 移到下一个 int 元素的位置
```

指针运算（арифметика указателей）主要只在同一个数组对象内部及其末尾后一个位置有效。

```
int a, b;
int *p = &a;
p++;
```

不能因为 `a`、`b` 声明相邻，就断言 `p == &b`。独立变量的内存排列没有这种保证，更不能随意解引用越过 `a` 后得到的指针。

第六章 数组与存储边界

当许多同类型数据需要统一处理时，逐个命名变量会使程序失去一般性。数组把这些对象按顺序存放，下标把一个整数映射到其中的元素。指针和数组由这种连续关系联系起来，但二者的类型、赋值规则和存储需求并不相同。

数组的连续存储与遍历

数组元素连续存放

```
int A[5] = {10, 20, 30, 40, 50};
```

```
        A[0]  A[1]  A[2]  A[3]  A[4]
       ┌─────┬─────┬─────┬─────┬─────┐
A ──→  │ 10  │ 20  │ 30  │ 40  │ 50  │
       └─────┴─────┴─────┴─────┴─────┘
```

在大多数表达式中，数组名 `A` 会转换为指向第一个元素的指针，即 `&A[0]`。但“数组就是指针”不严谨：

```
int A[5];  // 创建 5 个 int
int *p;    // 只创建一个指针变量
```

下标的本质

```
A[i] == *(A + i)
```

若 `int *p = A;`，则：

```
p[i] == *(p + i)
```

所以：

```
*p       == 10;
p[0]     == 10;
p[2]     == 30;
*(p + 2) == 30;
```

用指针遍历数组

下标版：

```
for (int i = 0; i < n; i++)
{
    A[i] = 10;
}
```

指针版：

```
for (int *p = A; p < A + n; p++)
{
    *p = 10;
}
```

`A + n` 是末尾后一个位置（указатель за последним элементом）。它可以用于比较和作为终点，但不能解引用：

```
int *end = A + n;  // 可以
*end = 10;         // 错误：越界访问，未定义行为
```

`p < A + n` 与 `p <= &A[n - 1]` 在 `n > 0` 时覆盖同一范围。第一种更自然，也适合表达空区间。

数组作为函数参数

在函数参数列表中：

```
void f(int a[])
void f(int *a)
```

参数会调整为指针类型，因此这两种写法等价。函数通常拿不到数组总长度，必须额外传入 n：

```
void fill(int a[], int n)
{
    for (int i = 0; i < n; i++)
    {
        a[i] = 10;
    }
}
```

俄语：

- 数组：массив
- 数组元素：элемент массива
- 下标/索引：индекс массива
- 遍历数组：обход массива
- 数组长度：длина массива
- 数组越界：выход за границы массива

sizeof 数组不等于 sizeof 数组参数

```
int A[] = {4, 7, 9};
size_t n = sizeof A / sizeof A[0];  // 这里得到 3

void fill(int A[], size_t n)
{
    for (size_t i = 0; i < n; ++i)
        A[i] = 0;
}
```

函数参数位置的 int A[] 会调整为 int *A，函数里的 sizeof A 得到指针大小，不能自动求出原数组长度。应单独传入 n，并保证 A 指向至少 n 个可写元素。

数组名在很多表达式中转换为首元素指针，但数组对象本身并不是指针变量：不能写 A = p；sizeof A 和 &A 也提醒我们二者类型和含义并不相同。

数组类型、指针类型与声明

设 int a[4] = {3, 8, 2, 6};，表达式 a 在大多数表达式中转换成首元素指针，类型为 int *；&a[0] 也指向首元素。&a 则是指向整个数组的指针，类型为 int (*)[4]。

它们通常显示相同的起始地址，但加 1 的步长不同：首元素指针前进一个整数，整个数组的指针前进整个四元素数组。类型决定指针运算所用的元素单位，不能只看打印出来的数值。

int *p[4] 是含四个指针的数组；int (*p)[4] 才是指向四元素数组的指针。括号改变声明结构。

```
int a[4] = {3, 8, 2, 6};
int *p = &a[1];
printf("%d %d\n", p[0], p[1]); // 8 2
```

`p[0]` 与 `*p` 等价；`p[1]` 与 `*(p + 1)` 等价。这里的下标以 `p` 当前指向的位置为起点，不一定以原数组 `a[0]` 为起点。

多维数组与行指针

二维数组可以看作“以一维数组为元素的数组”。声明 `int An2[3][5];` 建立三个元素，每个元素又是含五个 `int` 的数组。因此 `An2` 在多数表达式中转换为指向第一行的指针，而不是直接转换为 `int` 指针。

```
int An1[5] = {10, 20, 30, 40, 50};
int An2[3][5] = {{1, 2, 3, 4, 5},
                 {6, 7, 8, 9, 10},
                 {11, 12, 13, 14, 15}};
int *element = An1;
int (*row)[5] = An2;
```

`element` 指向一个整数，`row` 指向一个五元素数组。`element + 1` 越过一个 `int`；`row + 1` 越过一整行，即五个 `int`。步长由所指对象类型决定，即使两个指针经转换后代表同一个起始位置，也不因此获得相同的运算规则。

```
element = An2[1];     // 指向第二行的第一个整数
printf("%d\n", element[2]); // 8
row = &An2[1];        // 指向第二行这个数组对象
printf("%d %d\n", row[0][2], row[1][2]); // 8 13
row = &An1;           // 同为指向 int[5] 的指针
/* element = &An1; 类型不匹配 */
/* row = An1;       类型不匹配 */
```

`An2[1]` 的原始类型是 `int[5]`，在上述赋值中转换为 `int *`；`&An2[1]` 的类型则一直是 `int (*)[5]`。`row[1][2]` 先前进一行，再取那一行的第三个整数。若 `row` 已指向最后一行，就不能继续访问 `row[1]`。

把二维下标展开，可以看见两次不同的间接访问：`An2[i][j]` 等价于 `*(*(An2 + i) + j)`。第一次加法以行为单位，第二次以整数为单位。若每行有 `c` 个 `int`、`sizeof(int)` 为 `s`，元素相对整个数组起点的字节偏移为 `(i*c+j)*s`。这是内存布局的计算式；表达式本身仍须遵守各个数组对象的访问范围，不能把第一行的 `int` 指针任意跨行当成没有边界的一维数组。

二维数组作为函数参数

一维数组形参调整为首元素指针，二维数组也遵循同一规则，只是它的“一个元素”是一整行。设每行五个整数，下列两个声明表示同一种函数参数类型：

```
void zero_rows(int a[][5], size_t rows);
void zero_rows(int (*a)[5], size_t rows);
```

函数必须知道列数，才能由 `a + i` 找到第 `i` 行；行数另作为边界传入。形参中的第一维即使写成三，也不会把整个三行数组按值复制进函数，更不能替代运行时的边界检查。

讲解例：逐行清零。 下面给出完整调用，`<stddef.h>` 提供 `size_t`。

```
#include <stddef.h>
#include <stdio.h>

static void zero_rows(int a[][5], size_t rows)
{
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < 5; ++j)
            a[i][j] = 0;
}

int main(void)
{
    int a[3][5] = {{1, 2, 3, 4, 5},
                   {6, 7, 8, 9, 10},
                   {11, 12, 13, 14, 15}};
    zero_rows(a, 3);
    printf("%d %d\n", a[0][0], a[2][4]);
    return 0;
}
```

进入函数时，`a` 指向原数组第一行；`i` 从零增加到二，每次外循环把一行的五个元素改成零。函数没有建立第二份二维数组，所以调用后两个输出都是零。把三改成四会越界，函数签名不会自动发现实参只有三行。

`int **` 与这里的行指针不兼容。`int **` 指向的是一个“保存 `int` 指针的对象”，一次解引用应当读出一个指针；连续二维数组的一行里保存的却是整数。强制转换只改变解释方式，不会在整数前面凭空生成行地址表。

```
int r0[2] = {1, 2};
int r1[3] = {3, 4, 5};
int *rows[] = {r0, r1};
int **pp = rows;
```

这个新例中 `rows` 确实是指针数组，转换后可赋给 `pp`。`pp[1][2]` 先读取第二个行地址，再访问 `r1` 的第三个元素，结果为五。两行长度可以不同，甚至不相邻，函数因而需要分别知道各行长度。相同的双下标语法，背后可能是“固定步长定位”或“先读取地址”，必须由声明判断。

列数也要在运行时变化时，在支持变长数组参数的 C 实现中可以写：

```
void zero_matrix(size_t rows, size_t cols,
                 int a[rows][cols])
{
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < cols; ++j)
            a[i][j] = 0;
}
```

这里 rows、cols 应为正，实参应具有匹配的行宽和足够的行数；参数顺序使 cols 在后面的声明中已经可用。a 仍调整为“指向 cols 个 int 的数组”的指针，`sizeof a` 仍是指针大小。不能把此版本当作所有编译器都支持的固定数组版本；不支持这种参数类型的环境可采用前面的固定列数接口。另一个选择是从一开始就建立真正的一维缓冲区，通过 `i * cols + j` 表示二维索引，并检查长度与乘法溢出。它应与“把现成二维数组第一行的 int 指针任意跨行访问”区分开。

通用对象指针与地址比较

函数有时只负责保存或转交对象地址，暂时不需要知道对象类型，C 为此提供 void *。对象指针可转换为 void *，再转换回原类型；它不替程序保存原类型信息。标准 C 不允许对 void * 直接进行有意义的对象读取或指针算术，访问前必须恢复适当类型，并确保原对象仍存在。

```
void *pv1 = An1;
void *pv2 = &An1;
void *pv3 = &An1[0];
printf("%d\n", pv1 == pv2 && pv2 == pv3); // 1
int first = *(int *)pv1; // 10
```

这里三个地址都对应 An1 的起始位置，转换为 void * 后可比较相等。若第三个改成 `&An2[0]`，它指向另一个数组，便不应沿用“三个都相等”的结论。地址关系必须由实际声明和表达式确定，不能仅凭假设的地址数字判断。

数学中的连等关系不能直接写成 C 的连续比较。`pv1 == pv2 == pv3` 按结合性解析成 `(pv1 == pv2) == pv3`；内层比较得到 int 结果，而这个结果表达式不是空指针常量，不能再这样与指针比较。需要用 `&&` 连接两个合法比较。相似地，对普通整数写 `a < b < c` 也不会得到数学中的区间判断，应写 `a < b && b < c`。

下标与合法访问范围

```
/* 反例，假定 An 只有 20 个元素 */
An[100500] = 15;
```

三个概念一定分开：

部分	含义	俄语
An	数组，表达式中通常转换为首元素指针	массив
100500	相对 An 起点的元素下标，不是绝对地址	индекс
15	右侧要赋给目标元素的数值，不是地址推导结果	присваиваемое значение

15 来自赋值语句右侧。若换成 An[i] = i，才是把当前 i 的值写入 An[i]；不能因为下标或地址很大，就自动推导应写入 15。

地址公式只能在合法范围内用于理解

对于合法元素，概念上可用“首元素字节地址 + i × sizeof 元素”理解位置。例如假设起点地址 1000，int 占 4 字节，An[3] 位于 1012。这里只是教学假设，不是所有平台的实际地址。

对只有 20 个元素的数组访问 An[100500] 是越界（выход за границы массива）。即使按假设算出 403000，也不能说 C 保证会在那里写入 15：这样的程序已经出现未定义行为，可能崩溃、损坏其他数据、看起来正常，或被优化成意外结果。

负下标不总是同一回事

```
int A[] = {10, 20, 30};
int *p = &A[1];

printf("%d\n", p[-1]); // 合法：等同 A[0]，输出 10
/* A[-1] = 5; */      // 非法：跑到数组起点之前
```

判断标准是“计算后的指针是否仍在所属数组的合法范围”，不是看到负号就下结论。可以形成 A + 3 这个末尾后指针，用于终止比较；不可以读取或写入 *(A + 3)，也不可以继续形成 A + 4。

未定义行为与输出预测

未定义行为（неопределённое поведение，UB）意味着 C 标准不要求一个确定结果。它不同于“普通结果未知”，也不同于“程序一定报错”。

读题顺序：长度是否合法 → 下标是否在范围内 → 对象是否仍存活 → 类型是否匹配 → 最后才推算值与输出。UB 示例用于识别错误，不用于实际运行验证某个固定输出。

变长数组的长度约束

考虑一个根据运行时长度建立数组的函数：

```
/* 反例：未检查 n，不要以负数调用 */
void fun(int n)
{
    int An[n];
    for (int i = 0; i < n; ++i)
        An[i] = i;
}
```

An 的长度在运行时决定，属于变长数组（массив переменной длины，VLA）。在支持这一特性的 C 实现中，每次运行到这个声明时，长度必须大于 0。

n = -5 时，i = 0，条件 0 < -5 是假，所以循环体不执行；但数组声明已经在循环前执行。不执行循环不能消除这个声明的问题。编译通过、运行没有报错、不一定立刻崩溃，都不能证明程序合法。

先检查，再声明

```
/* 仅适用于支持 VLA 的编译器 */
void fun(int n)
{
    if (n <= 0 || n > 1000)
        return;
    int An[n];
    for (int i = 0; i < n; ++i)
        An[i] = i;
    printf("%d\n", An[n - 1]);
}
```

这里的 1000 是示例选择的容量上限，不是 C 标准规定的最大数组长度。上限需要结合元素大小与可用资源决定；只检查正数还不够防止过大的自动数组。

微软 MSVC 不支持这种 VLA 写法。若课堂使用 Visual Studio 的 MSVC，可先使用固定容量方案：

```
void fun_fixed(int n)
{
    if (n <= 0 || n > 1000)
        return;
    int An[1000];  // 实际容量固定，只使用前 n 个元素
    for (int i = 0; i < n; ++i)
        An[i] = i;
    printf("%d\n", An[n - 1]);
}
```

C 中 const int n = 10 也不自动成为整数常量表达式；别把 C++ 的数组长度规则直接搬来。固定编译期长度可以使用 enum { CAPACITY = 1000 };。编译器支持情况见 Microsoft 的 C11/C17 支持说明。

第七章 类型限定与数值表示

一个地址本身不足以说明可以对内存做什么。类型限定约束写入权限，运算规则约束可进行的计算，而数值表示决定哪些数学结果能够被精确保存。本章把这些条件与前面读写对象的操作联系起来。

const 限定与指针比较

const 相关内容是围绕学习中的问题补充的系统说明。俄语：константный указатель（常量指针）；указатель на константу（指向常量的指针）。

先分开两个问题

1. 能不能让 p 改为指向另一个对象？ 2. 能不能通过 *p 修改当前对象？

声明	p 可以改指向？	可以通过 *p 写值？
int *p	可以	可以

声明	p 可以改指向？	可以通过 *p 写值？
const int *p	可以	不可以
int const *p	可以	不可以
int *const p = &x	不可以	可以
const int *const p = &x	不可以	不可以

const 在 * 左边，限制通过这个指针访问的对象；const 在 * 右边，限制指针变量本身。常量指针必须在定义时给定指向。

```
int x = 3, y = 8;
const int *p = &x;
p = &y;          // 允许
/* *p = 5; */    // 不允许：通过 p 写值
x = 6;           // 允许：x 本身不是 const 对象

int *const q = &x;
*q = 7;          // 允许：x 可修改
/* q = &y; */    // 不允许：q 不可改指向
```

const int *p 不代表它所指的一切值永远不变，只表示这个访问通道不允许写。如果对象本身定义为 const int x = 3，强制去掉 const 再修改它是未定义行为，不能靠类型转换绕过限制。

指针相等、大小比较和值比较

p == q 问“是否指向同一位置”；*p == *q 问“两个对象的值是否相等”。两个不同变量即使都存 5，它们的指针也不因此相等。

*p < *q 可以用于有效整数对象的值排序；p < q 的可移植用法则限于标准规定的关联对象范围，例如同一数组中的元素。不要把四个独立变量的地址大小当成值大小，也不要用地址顺序推断声明顺序。

浮点数表示与数值比较

浮点数通常不能精确表示所有十进制小数，例如 1/3。比较近似计算结果时，应根据问题尺度选择绝对或相对容差，而不是把一个固定 0.0001 当成适合所有问题的规则。

```
#include <math.h>
int close_enough(double a, double b,
                 double abs_tol, double rel_tol)
{
    return fabs(a - b) <= abs_tol + rel_tol * fmax(fabs(a), fabs(b));
}
```

这是有限普通数值的课堂示例，未把 NaN 和无穷的完整策略纳入。fabs 接受浮点数；整数绝对值与浮点绝对值函数不是同一个接口。

地址用 %p 配 (void *)p 输出。不要把地址保存进 int 并假定不会截断。uintptr_t 是可选的无符号整数类型，若实现提供它，可用于相应指针与整数转换；它的存在也不使任意整数地址成为可安全访问的对象。

第八章 字符串与字符指针

字符串在 C 中以字符数组保存，用零字符标记结尾。这种表示使字符串可以沿用数组和指针的操作，同时要求程序始终明确字符内容和存储容量的区别。

字符编码与零结尾表示

char 保存字符编码。ASCII 环境下，'A' 的值为十进制 65、十六进制 41；这三个写法表达同一个数值。输出格式决定显示方式，不能由此认为所有 C 实现都必须使用 ASCII。

```
char ch = 'A';
printf("%c %d %x\n", ch, (int)ch, (unsigned int)(unsigned char)ch);
/* ASCII 环境：A 65 41 */
```

%c 输出一个字符，%s 从给定地址读取字符直到第一个零字符。'\0' 是数值为零的字符；'0' 是数字零的字符，两者不同。'\n' 是一个换行字符，虽然代码中用两个可见符号书写，但只占一个 char 元素。

```
char a[] = "ABC";                 // 4 元素，末尾有 '\0'
char b[] = {'A', 'B', 'C'};       // 3 元素，没有结束符
char c[] = {'A', 'B', 'C', '\0'}; // 4 元素，是字符串
char d[3] = "ABC";               // C 中合法，但没有 '\0'
char e[10] = {'A', 'B', 'C'};     // 其余元素补零，是字符串
char Mes[] = {'\n', 'A', 'B', 'C', 'D', '\0'};
```

字符数组是否带有结束符，取决于初始化方式与数组容量。字符串字面量自带结束符，但定长字符数组恰好只能容纳普通字符时可以省去它；逐项初始化时，未显式给值的剩余数组元素补零，完全填满的数组不会额外增加一个元素。因此 b 和 d 不能交给 %s 或 strlen；它们仍然是合法字符数组，却不是零结尾字符串。

Mes 与 char Mes[] = "\nABCD"; 的内容相同。输出时使用 printf("%s", Mes);，不要把可变内容直接放在格式参数位置；否则内容中的 % 会被解释成格式说明。

字面量、变量与字符存储

程序中的 10、'A'、"ABC" 直接给出了固定的值或字符序列，通常称为字面量；C 标准将 'A' 称为字符常量。变量名则标识保存数据的对象。char a[] = "ABC"; 用字面量初始化新的字符数组，const char *p = "ABC"; 只建立一个指向字面量的指针，两种初始化产生的对象不同。

字符数组与字符串字面量

```
char str1[10] = "ABC";
char str2[] = "DEF";
const char *str3 = "GHI";

str1[1] = '_';
str2[1] = '_';
/* str3[1] = '_'; 约束违反，编译器须给出诊断 */
printf("%s %s %s\n", str1, str2, str3);
/* A_C D_F GHI */
```

str1 和 str2 自己拥有可写字符存储；str3 是保存地址的指针。const 禁止通过它写字符。在 C 中，即使写成 `char *p = "GHI";`，也不能修改字面量：尝试写入会产生未定义行为。现代标准 C++ 不允许用字符串字面量直接初始化 char *，通常会在编译时拒绝此声明。C 与 C++ 的规则不能混用。指针能否改指向，与所指字符能否修改，是两个问题。

字符串指针数组与双下标

```
char *sarr[] = {str1, str2};
const char *sarr2[] = {str1, str2, str3, "banana"};
printf("%s %s\n", sarr[1], sarr2[3]); // D_F banana
printf("%c\n", sarr[1][2]);          // F
sarr2[0] = "hello";                 // 可以改保存的地址
/* sarr2[0][0] = 'X'; 不允许通过 const char * 写字符 */
```

sarr 是数组，每个元素为 char 指针。`sarr[1]` 取第二个指针；`sarr[1][2]` 再取该指针指向位置之后的第三个字符，等价于 `*(*(sarr + 1) + 2)`。它不同于 `char (*p)[5]`：后者是一个指向五元素字符数组的指针。

字符串长度与对象大小

strlen 声明在 `<string.h>`，返回第一个结束符之前的字节数，不包含结束符。它要求输入已经是有效字符串，不能用它先判断一个无结束符数组是否安全。sizeof 是运算符，计算对象或类型的字节大小。

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char str1[10] = "ABC";
    char str2[] = "DEF";
    char Mes[] = "\nABCD";
    printf("%zu %zu %zu\n", strlen(str1), strlen(str2), strlen(Mes));
    printf("%zu %zu %zu\n", sizeof str1, sizeof str2, sizeof Mes);
    // 两行分别为：3 3 5 和 10 4 6
    str1[1] = '\0';
    printf("%zu %zu\n", strlen(str1), sizeof str1); // 1 10
    return 0;
}
```

两者结果类型都是 size_t，输出用 %zu。对 const char *p = "GHI";，strlen(p) 为 3，sizeof p 是指针自身大小，不能推断为 4，也不能据此获得目标容量。函数参数中的数组声明会调整为指针，同样不能在函数内用 sizeof 参数求原数组容量。

示例中的 ls 是小写字母 l 和 s，不是数字 1 加 s，也不是关键字。它只是长度变量，规范写法为 size_t ls = strlen(str2);。中文等多字节编码下，strlen 计算字节数，不等同于人所理解的汉字个数。

第九章 字符串处理函数

把字符串交给函数处理之前，必须先约定源数据的结束位置和目标的可写范围。复制函数并不拥有数组，它只能依照调用者给出的地址与长度读写内存。

字符串的逐字符复制

strcpy(dst, src) 从源复制到目标，包含结束符。前提是源为有效字符串、目标可写且容量至少为 strlen(src) + 1，源目标区域不能重叠。

```
char dst[10] = "ABC";
char src[] = "D_F";
strcpy(dst, src);
printf("%s\n", dst); // D_F
```

实际写入 D、_、F、零字符，共四个字节。数组剩余位置不会被该函数自动清零。不能写 dst = src 给整个数组赋值；若两边都是指针，赋值只是改变地址，也不会复制字符串。

从下标循环到指针循环

复制两个数组的内容，归根结底是依次读取源元素、写入目标元素。对于字符串，循环还要识别结束符，并把这个结束符也写入目标。先不调用库函数，写出最直接的实现：

```
/* 前提：src 是有效字符串，dst 容量足够，区域不重叠 */
void copy_text(char *dst, const char *src)
{
    size_t i = 0;
    for (;;) {
        dst[i] = src[i];
        if (src[i] == '\0')
            break;
        ++i;
    }
}
```

若源为 "CAT"，i 为 0、1、2 时依次写入 C、A、T；i 为 3 时先写入零字符，再退出。检查发生在写入之后，所以结束符不会遗漏。这里的函数名使用 copy_text，以免同标准库 strcpy 的声明冲突。

下标只是指针加法与解引用的另一种写法。因此，也可以用两个指针记录当前位置：

```
void copy_text_pointer(char *dst, const char *src)
{
    for (;;) {
        *dst = *src;
        if (*src == '\0')
            break;
        ++dst;
        ++src;
    }
}
```

两个函数做同一件事：前者改变下标，后者改变局部指针。形参指针本身按值传入，函数内移动 dst 不会让调用者的指针跟着移动；通过 *dst 写入的字符则属于调用者提供的数组。由此，值传递和间接修改在同一段程序里同时出现，并不矛盾。

进一步把赋值结果用于条件，可以写成 `while ((*dst++ = *src++) != '\0') { }`。后缀自增使本次读写仍使用原地址，随后两个局部指针各前进一个字符；最后复制零字符，比较结果为假，循环结束。这个短写法需要从完整版本逐步理解，不能仅凭外形背诵。

上述实现都没有目标容量参数，不能自行检查可写空间。库函数 strcpy 也有相同前提。代码行数变少不会使容量问题消失，结束符约定和数组边界仍然是调用者必须满足的条件。

定长复制与零填充

`strncpy(dst, src, n)` 最多从源取 n 个字符；若先遇结束符，则用零填满目标的这 n 个位置。若源的前 n 个字符都不是零，目标不会自动追加结束符。目标必须有至少 n 个可写位置，区域仍不得重叠。

```
char dst[10] = "ABCDEFG";
char src[] = "DEF";
size_t ls = strlen(src); // 3
strncpy(dst, src, ls);
printf("%s\n", dst);   // DEFDEFG：旧尾部仍在
dst[ls] = '\0';
printf("%s\n", dst);   // DEF
```

这里手工补零合法，因为 ls 小于目标容量；不能在任意长度下照抄。给源数组 `src[ls]` 写零不会替目标补结束符。手工补结束符的下标必须小于目标容量，否则补零本身也是越界写入。

下面例子先检查容量，再完整复制；空间不足时保留原目标，不进行截断：

```
size_t ls = strlen(src);
if (ls < sizeof dst) { // 这里 dst 必须确实是数组对象
    strcpy(dst, src);
} else {
    puts("destination too small");
}
```

strcpy_s、strncpy_s 与微软扩展

在支持相应接口的环境中，`strcpy_s(dst, capacity, src)` 有三个参数；微软的 `strncpy_s(dst, capacity, src, count)` 有四个。两者的参数个数和复制范围不同。`_countof(array)` 是微软提供的数组元素个数宏，不是通用标准 C 写法，不能用指针推算原数组容量。

```
/* 以下为 Microsoft CRT 环境示例 */
char dst[10] = "ABC";
errno_t err = strcpy_s(dst, _countof(dst), "DEF");
if (err != 0) {
    /* 按项目要求处理复制失败 */
}
```

容量必须如实描述目标可写范围，并为结束符留位置。微软 CRT 某些参数错误会先触发无效参数处理器，默认行为可能终止程序，不能假定任何错误都会正常返回到 if。标准 C 的 Annex K 接口是可选部分，可用性和配置因实现而异；跨平台示例优先明确管理容量，不能假设所有编译器都提供上述函数。

内容比较、拼接与整行输入

地址相等只说明两个指针指向同一位置，不说明两个不同数组保存的文字是否相同。`strcmp` 比较零结尾字符串的内容，返回负、零、正分别表示前者按字节比较较小、相等、较大；标准不要求恰好返回负一或一。

```
char first[] = "cat";
char second[] = "cat";
printf("%d\n", strcmp(first, second) == 0); // 1
```

两个数组是不同对象，但字符依次相同。比较按第一个不同字符的 unsigned char 值决定，不是按字符串长度决定，也不是自然语言词典排序。`strncmp(a, b, n)` 最多比较 n 个字符，因此前 n 个字符相同不能推出整个字符串相同。例如 ″cart″ 与 ″carpet″ 的前三个字符相同。

拼接在目标字符串原有结束符处开始写入。若使用 `strcat(dst, src)`，必须已有合法目标字符串，容量至少为 `strlen(dst) + strlen(src) + 1`，源与目标不得重叠。`strncat` 的 n 限制的是最多从源复制多少字符，仍会另外写结束符，n 也不是目标总容量。设目标容量八，原内容 ″ab″，再接 ″cde″ 需六个元素；若继续接 ″fgh″，总需求九个，已经超出容量，不能因为每次只加三个就认定安全。内容、已用长度和总容量始终是三个量。

读取含空格的整行时，可以让 `fgets` 接收缓冲区容量：

```
char line[80];
if (fgets(line, sizeof line, stdin) != NULL) {
    size_t p = strcspn(line, "\n");
    int complete = line[p] == '\n';
    line[p] = '\0';
    printf("%s\n", line);
    /* complete 为 0 时，仍须判断是否读到了完整一行。 */
}
```

成功读取时，函数至多保存七十九个输入字符，再补结束符；读到换行且有空间时，换行也会保存。`strcspn` 找到首个换行或原结束符的位置，随后置零可去掉已读到的换行。但缓冲区里没有换行，可能是最后一行没有换行，也可能是一行过长、剩余部分还在输入流中。二者不能混为“成功读完整行”，应按应用要求继续读取、丢弃余段或报告过长。

字符串函数普遍依赖有效对象和容量条件；它们不能修复已经没有结束符的输入。若要移动可能重叠的字节区间，应使用按字节长度操作的 `memmove`，而不是 `strcpy` 或要求不重叠的 `memcpy`。字节搬运本身不自动补字符串结束符。关于参数调整与标准库语义，可核对 WG14 C11 委员会草案 N1570 的 6.7.6.3、7.21.7.2 和 7.24；它是语义参考，本册讲解仍按对象、边界、操作过程展开。

第十章 交换与按位运算

排序把一组对象按比较关系排列起来。实现它之前，需要分清是移动整数值，还是改变指向这些值的指针。交换操作确定以后，才能准确描述一轮排序改变了什么、保持了什么。排序之后，中位数可以直接由位置确定；如果不能排序，也可以从元素的秩出发寻找它。

间接修改与指针形参

交换两个变量的值

```
void swap_values(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

调用 `swap_values(&x, &y);` 后，`x` 和 `y` 的值互换。

交换两个指针保存的地址

```
int *pt = pb;
pb = pa;
pa = pt;
```

若最初：

```
a = 35，pa → a
b = 8，  pb → b
```

交换后：

```
a 仍为 35，b 仍为 8
pa → b，所以 *pa == 8
pb → a，所以 *pb == 35
```

交换指针不等于交换变量里的值。

如果要在函数中修改调用者的指针变量本身，需要二级指针：

```
void swap_pointers(int **p1, int **p2)
{
    int *temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}
```

调用：

```
swap_pointers(&pa, &pb);
```

不要为了“不使用第三个变量”而对指针做异或交换。标准 C 不允许直接对指针使用 ^；把指针转为整数再异或也会降低可移植性和可读性。临时指针是清楚、安全的常规写法。

相关俄语作业表述：

- `4 переменных`：4 个变量。
- `сортировка по возрастанию`：升序排序。
- `сортировка по убыванию`：降序排序。
- `обмен указателей`：交换指针。
- `без использования третьей переменной`：不使用第三个变量。
- `x86 / x64`：常用于讨论 32 位/64 位环境。指针大小应用 `sizeof(void *)` 判断，不要写死。

对象交换与地址交换

交换两个整数值，需要两个整数对象的地址：

```
void swap_values(int *pv, int *qv)
{
    int t = pv[0];
    pv[0] = qv[0];
    qv[0] = t;
}
```

调用 `swap_values(&a[0], &a[3]);` 会改变 a 中的值。数组下标同样是间接访问；`pv[0]` 与 `*pv` 访问的是同一个整数对象。

交换相邻值，只传一个起始指针也可以，但必须确认后面还有一个有效元素：

```
void swap_neighbors(int *pv)
{
    int t = pv[0];
    pv[0] = pv[1];
    pv[1] = t;
}
```

调用 `swap_neighbors(&a[1]);` 交换 a[1] 与 a[2]。不能把 a[3] 的地址传给它并访问 pv[1]，因为那会越界。

交换指针数组中的地址，临时变量应是 `int *`：

```
void swap_addresses(int *p[], int i, int j)
{
    int *temp = p[i];
    p[i] = p[j];
    p[j] = temp;
}
```

这改变 p 的访问顺序，不改变 a 中整数的排列。看函数时不要只看名字 swap，要看赋值号左边究竟是 `p[i]` 还是 `*p[i]`。

按位异或与整数运算边界

^ 不是乘方，也不是逻辑或

按位异或（побитовое исключающее ИЛИ，XOR）逐位比较整数：不同为 1，相同为 0。

第一位	第二位	异或结果
0	0	0
0	1	1
1	0	1
1	1	0

```
5       0101
3       0011
5 ^ 3   0110  → 6
```

规则：x ^ x = 0，x ^ 0 = x，(a ^ b) ^ b = a。这是按位运算的性质，不是地址交换的理由。

C 表达式务必写清括号：(5u ^ 3u) == 6u 为真。若写 5u ^ 3u == 6u，== 比 ^ 优先级高，会被解释为 5u ^ (3u == 6u)，含义已经不同。

三步整数交换

```
unsigned int a = 5u, b = 3u;
a ^= b;    // a=6, b=3
b ^= a;    // a=6, b=5
a ^= b;    // a=3, b=5
```

第二步得到 b_old ^ (a_old ^ b_old) = a_old；第三步再次抵消原 a，恢复原 b。其依据在于取消重复的位，而不是“神奇地交换”。

如果通过两个指针做异或交换，两个指针不能指向同一个整数：第一次 *p ^= *q 就会把它清零。一般代码直接使用临时变量，含义更清楚；不要把技巧当成更好的默认写法。

整数溢出与模运算

数学上 a=a+b; b=a-b; a=a-b 可交换整数；但有符号整数的 a+b 若超出可表示范围，会出现未定义行为。不能因为后两步能“减回来”，就忽略第一步的溢出（переполнение）。

无符号整数按模运算回绕，不能一概说“回绕就一定交换失败”。以恰好 8 位的无符号存储为教学例：250+10 回绕为 4；4-10 回绕为 250；4-250 回绕为 10，最终仍可交换。本例是模 256 的推导，不代表 unsigned int 在所有平台都是 8 位。

指针运算的类型与范围条件

运算	是否可以	前提
p + k、p - k	可以	结果在同一数组内或末尾后一位置
p - q	可以	同一数组或末尾后一位置；差可表示
p + q	不可以	C 不定义两个指针相加
p ^ q	不可以	^ 要求整数操作数
*p ^ *q	可以	p、q 指向有效的整数对象

指针差的类型是 ptrdiff_t，头文件 <stddef.h>，printf 可用 %td。差表示相隔的元素数，不是字节数。

把指针强转为整数再异或，不是可移植的指针交换办法。uintptr_t 是可选类型；它支持的是符合规定的指针原值往返转换，不保证任意异或后的中间整数转成指针再转回来仍保持位模式。不要把这种写法收入“推荐代码”。

第十一章 伪随机数与排序

排序算法需要把比较结果转化为数据移动。以一个固定数组追踪每轮的变化，可以先检查算法本身；需要批量产生测试数据时，再使用伪随机数生成器。

伪随机数与区间映射

```
#include <stdlib.h>
#include <time.h>
```

rand() 是伪随机

rand() 生成伪随机数（псевдослучайное число）：

```
0 <= rand() <= RAND_MAX
```

RAND_MAX 是 <stdlib.h> 中的常量。C 标准保证它至少为 32767，具体值由实现决定：

```
printf("%d\n", RAND_MAX);
```

种子决定序列从哪里开始

```
void init_rng(void)
{
    srand((unsigned int)time(NULL));
}
```

- 种子：seed；俄语常说 начальное значение 或 зерно генератора。
- 相同实现中，相同种子产生相同的伪随机序列。
- 当前时间不断变化，因此常用作简单程序的种子。
- 一般在程序开始时初始化一次，不要每生成一个数就重新 srand。

```
int main(void)
{
    init_rng();
    int a = rand();
    int b = rand();
}
```

调用并丢弃开头几个 rand() 不是必要规则，也不能从根本上提高随机质量。

缩放到 0～1000

```
int gen_rnd_int(void)
{
    long long t = ((long long)rand() * 1000) / RAND_MAX;
    return (int)t;
}
```

1000 只是例子选择的上限，不是 rand() 的规定。

在进行乘法前先转换为 long long，是为了扩大中间结果的表示范围。

```
(long long)rand() * 1000
```

这样乘法在较宽的整数类型中进行，降低 int 乘法溢出的风险。转换必须在乘法之前；若写成 (long long)(rand() * 1000)，内部乘法可能已经溢出。

边界：

- rand() == 0 → 结果为 0；
- rand() == RAND_MAX → 结果为 1000。

所以该公式包含 1000。

```
rand() % 1000
```

会得到 0 999，但可能存在模偏差（modulo bias）。对分布要求不高的演示可以采用这种写法；安全、密码或高质量统计用途不能使用 rand()。

指针数组的间接排序

题目要按升序（по возрастанию）读取四个值，同时保持 a、b、c、d 本身不变。应比较所指的值，再交换指针保存的地址，而不是交换 *pa 与 *pb。

```
#include <stdio.h>

void order_pair(int **p, int **q)
{
    if (**p > **q)
    {
        int *tmp = *p;
        *p = *q;
        *q = tmp;
    }
}

int main(void)
{
    int a = 35, b = 8, c = 22, d = 4;
    int *pa = &a, *pb = &b, *pc = &c, *pd = &d;

    order_pair(&pa, &pb);
    order_pair(&pc, &pd);
    order_pair(&pa, &pc);
    order_pair(&pb, &pd);
    order_pair(&pb, &pc);

    printf("original: %d %d %d %d\n", a, b, c, d);
    printf("sorted: %d %d %d %d\n", *pa, *pb, *pc, *pd);
    return 0;
}
```

预期输出：

```
original: 35 8 22 4
sorted: 4 8 22 35
```

二级指针的接口作用

调用 &pa 传的是“指针变量 pa 自己的地址”。函数内 *p 就是 pa，p 才是 pa 所指整数。比较 p > **q 是比较数值；赋值 *p = *q 是改调用者的指针，而不是改 a、b、c、d。

次序	比较位置	比较后按 pa、pb、pc、pd 读取
初始	-	35、8、22、4
1	pa、pb	8、35、22、4
2	pc、pd	8、35、4、22
3	pa、pc	4、35、8、22
4	pb、pd	4、22、8、35
5	pb、pc	4、8、22、35

前两步各自整理两个小组；随后把最小值送到第一位、最大值送到第四位；最后整理中间两位。因为比较使用 >，相等时不交换本次所比较的两个指针；但不能因此声称整个网络是稳定排序。

算法实现与接口约束

上面的可读版本使用临时指针 tmp 和辅助函数；若题目明确禁止新增变量、禁止函数或禁止交换，这个版本不满足该额外限制。它用于解释正确目标和标准做法，不能标成“满足所有限制的答案”。应先确认题目到底允许重绑定指针、条件表达式还是只禁止某一种 swap 写法，再选择策略。禁止临时变量不等于允许对指针做非法算术。

选择排序及循环不变量

从大到小排序时，第 i 轮在尚未排列的区间 i…n-1 中找最大值下标，再与第 i 个位置交换。外层循环固定“这一轮排好哪里”，内层循环负责“在剩余部分找谁最大”。

循环不变量：第 i 轮开始前，前 i 个位置已经按降序放好了最大的 i 个值。完成这一轮后，这个性质扩展到 i+1 个位置。这个解释说明算法为何正确，而不仅是解释每行语法。

小例子。 3、8、2、6：第 0 轮找到 8，交换后是 8、3、2、6；第 1 轮从 3、2、6 中找 6，得到 8、6、2、3；第 2 轮得到 8、6、3、2。最后一个数自然就位，所以只需 n-1 轮。

下面是用指针数组实现降序访问的完整课堂程序。注释和输出使用英文。随机数缩放沿用课堂公式，它适合演示，不保证 0…1000 每个值严格等概率。

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int gen_rnd_int(void)
{
    return (int)(((long long)rand() * 1000) / RAND_MAX);
}

void swap_addresses(int *p[], int i, int j)
{
    int *temp = p[i];
    p[i] = p[j];
    p[j] = temp;
}

int main(void)
{
    int a[100];
    int *p[100];
    int i, j, max_index;
    double median;

    srand((unsigned int)time(NULL));
    printf("Original numbers:\n");
    for (i = 0; i < 100; i++) {
```

```
        a[i] = gen_rnd_int();
        p[i] = &a[i];
        printf("%d ", a[i]);
        if ((i + 1) % 10 == 0)
            printf("\n");
    }

    // Sort the pointers by the values they refer to
    for (i = 0; i < 99; i++) {
        max_index = i;
        for (j = i + 1; j < 100; j++) {
            if (*p[j] > *p[max_index])
                max_index = j;
        }
        swap_addresses(p, i, max_index);
    }

    printf("Descending order:\n");
    for (i = 0; i < 100; i++) {
        printf("%d ", *p[i]);
        if ((i + 1) % 10 == 0)
            printf("\n");
    }
    median = *p[49] / 2.0 + *p[50] / 2.0;
    printf("Median: %.1f\n", median);
    return 0;
}
```

`max_index` 保存位置，不保存最大值本身；比较的是 `*p[j]`，不是两个地址谁更大。`(i+1)%10==0` 表示已经打印的数量是 10 的倍数，此时换行。

如果改成直接交换 a 中的值，比较和交换对象都要一致替换；不要留下指针版的一半逻辑，又混入数值版的一半逻辑。

选择排序的状态变化

`max_index` 记录的是数组位置。它不能一会儿保存下标、一会儿保存数值，否则后面的 `a[max_index]` 就可能把普通数值误当作下标。以数组 3、8、2、6 的第一轮为例，固定 i=0，内层搜索过程为：

已检查的位置	当前比较	max_index	已检查部分的最大值
初始化	把当前位置作为候选	0	3
j=1	8 大于 a[0]	1	8
j=2	2 不大于 a[1]	1	8
j=3	6 不大于 a[1]	1	8

搜索过程中数组没有改变。只有内层循环结束后，才交换 a[0] 与 a[1]，把这一轮已经确认的最大值固定到前端。第二轮 i=1 时，`max_index = i` 重新把未排序区间的第一个位置作为候选；不能沿用上一轮的候选位置，因为上一轮的元素已经进入排好部分。

内层从 j=i+1 开始，是因为 a[i] 已经作为候选，不必再同自己比较；内层必须扫描到 n-1，才能保证没有漏掉更大值。外层只到 i=n-2，因为前 n-1 个位置确定后，剩下最后一个元素已经无处可选。

同一算法的两种存储方案

直接排序整数数组时，比较对象是整数，交换对象也是整数。下面的函数使用下标形式访问指针指向的对象，能够交换不相邻位置：

```
void swap_values(int *left, int *right)
{
    int temp = left[0];
    left[0] = right[0];
    right[0] = temp;
}

void selection_sort_desc(int a[], int n)
{
    int i, j, max_index;
    for (i = 0; i < n - 1; ++i) {
        max_index = i;
        for (j = i + 1; j < n; ++j) {
            if (a[j] > a[max_index])
                max_index = j;
        }
        swap_values(&a[i], &a[max_index]);
    }
}
```

若调用前 a 是 3、8、2、6，函数结束后 a 自身变成 8、6、3、2。相反，指针数组方案先令 `p[i] = &a[i]`；比较 `*p[j]` 与 `*p[max_index]`，再交换 p 中的两个地址。此时 a 仍保持 3、8、2、6，只有依 p 访问时才读到 8、6、3、2。两种程序共享“在剩余部分找最大值”的算法，但保存排序结果的位置不同。

若只传入一个指针 pv，函数内交换 `pv[0]` 与 `pv[1]`，它只能操作当前位置及紧邻的下一个元素。选择排序找到的最大值可能距离当前位置很远，因此应传两个地址，或者把数组和两个下标传进去，不能把单指针的相邻交换函数原样替代上述交换。

程序变量与显示格式

`int a[100];` 建立一百个整数对象；`int *p[100];` 建立一百个指针对象。局部声明本身不会生成随机数，也不会使 p 自动指向 a。必须先给 a[i] 写入数值，再按需要让 p[i] 保存对应地址，之后才能读取它们。i、j 是循环计数器，max_index 是候选位置，median 是最后的统计量，它们承担的角色各不相同。

每行显示十个数时，下标 i 与已打印数量相差一。i 为 9 时刚打印第十个数，所以条件是 `(i + 1) % 10 == 0`；若写 `i % 10 == 0`，第一个数刚输出就换行，版面会错位。这个条件只控制显示，不参与排序或中位数计算。

示例开头的 `_CRT_SECURE_NO_WARNINGS` 是 Microsoft CRT 相关宏，用于抑制一类运行库弃用提示，不是 `#nowarning` 这样的 C 指令，也不改变数组边界或函数语义。涉及该运行库头文件时，宏应在包含头文件之前定义。其他工具链不需要靠这个宏才能使用标准 C 语法。

平均值与整数除法

若两个中间值为 20、21，表达式 `(20 + 21) / 2` 的两个操作数都是整数，结果是 20；随后赋给 double 也只能得到 20.0。写成 `(20 + 21) / 2.0`，才在除法之前引入浮点运算，得到 20.5。`%.1f` 决定显示一位小数，并不会把一个已经截断的整数结果恢复为正确平均值。

对本例 0-1000 的输入，两数相加不会超过通常使用的 int 范围；若函数后来处理任意 int，则应避免先做可能溢出的整数加法。本册用 `x / 2.0 + y / 2.0` 表达平均值，先转换再运算。能否准确表示任意大整数还取决于 double 的精度，不能从这段课堂数据范围推广为无限精确。

第十二章 中位数与顺序统计

中位数是按大小次序选出的位置统计量。它由秩决定，而不是由原数组的存储顺序决定。排序是求中位数的一种实现方式，但中位数的定义不依赖是否真的执行排序。

顺序统计量与中位数

对 100 个数，中间两个次序统计量是第 50 小和第 51 小；数组下标从 0 开始，对应 49、50。平均值可能为半整数，因此用 double。

不排序时，对每一个候选值 v 统计：L 个数严格小于 v，E 个数等于 v。v 在升序中的位置覆盖 L+1…L+E，因此它是第 k 小的充要条件为：

$$L < k \le L + E\,.$$

这个区间能正确处理重复值。例如 2、2、2、9，候选值 2 的 L=0、E=3，所以第 1、2、3 小都等于 2。不要只统计小于数而忽略重复数。

完整的不排序程序如下，它保留原数组：

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    int a[100];
    int i, j, less, equal;
    int x = 0, y = 0;
    double median;

    srand((unsigned int)time(NULL));
    printf("Original numbers:\n");
```

```
    for (i = 0; i < 100; i++) {
        a[i] = (int)(((long long)rand() * 1000) / RAND_MAX);
        printf("%d ", a[i]);
        if ((i + 1) % 10 == 0)
            printf("\n");
    }

    // Find ranks 50 and 51 without sorting
    for (i = 0; i < 100; i++) {
        less = 0;
        equal = 0;
        for (j = 0; j < 100; j++) {
            if (a[j] < a[i])
                less++;
            else if (a[j] == a[i])
                equal++;
        }
        if (less < 50 && less + equal >= 50)
            x = a[i];
        if (less < 51 && less + equal >= 51)
            y = a[i];
    }
    median = x / 2.0 + y / 2.0;
    printf("Median: %.1f\n", median);
    return 0;
}
```

虽然 x 和 y 都由 a[i] 赋值，每一轮的 i 却不同，而且两个 if 的条件也不同。某个候选值可能只覆盖第 50 小，另一个覆盖第 51 小；若两个位置恰好落在同一重复值区间，也可以得到相同的 x、y。

这个算法时间复杂度为 $O(n^2)$，适合当前知识范围；不排序并不自动意味着比排序快。更高效的选择算法可以以后再学。

附录 术语与阅读资料

变量对应俄语 переменная，值为 значение，地址为 адрес，指针为 указатель，数组为 массив，函数为 функция，实参为 аргумент，循环为 цикл，递归为 рекурсия。指针解引用称为 разыменование，升序排列为 по возрастанию。

K&R《C 程序设计语言》第二版：第 1 章提供由小程序引入语言机制的教学顺序；第 2-4 章讨论表达式、控制流和程序结构；第 5 章将地址、指针、数组和字符处理结合起来。所用本地 PDF 的第 21-23、41-43、96-108、112-113 页已读取相关正文。原书中的隐式 int 和旧式函数示例应结合现代编译器要求辨别，不能直接当作当前推荐写法。

程序语义参照 WG14 N1570；Microsoft CRT 的 *_s 与 _countof 依工具链文档核查。完整程序、函数片段和反例已分别标明；片段需配合头文件与调用者。本册示例仅经逻辑审阅，未编译运行。